Práctica 4 Realizada por Robert Mosquera Rocamora 2 DAM O.



Tarea a realizar:..... 1
Configurar API 2
Fromulario..... 4
Enrutado 6
Navegación entre pantallas..... 7
NavigationBar 8
Ampliación de la práctica 9
Navigation bar en paises 10
Orientación de pantalla 10

Tarea a realizar:

Terminar de implementar las funciones necesarias en PaisesGUI:

Navegación entre páginas.

Usar la nueva api junto con la antigua.

Formulario validado.

Usar navigationBar

Configurar API

```
late PaisesEjemplo api;
late PaisesEjemploNuevo apiNueva;

PaisesRepository() {
  api = PaisesEjemplo();
  apiNueva = PaisesEjemploNuevo();
}
```

Creamos y añadimos la nueva api

```
//Practica 4

 List<Continente> getContinentesNuevo() {
      // Devuelve una lista de provincias, obtenidas a partir de la propiedad
  // provincies de RepositoryData.

  List<Continente> continentes = [];
  for (var conti in apiNueva.continentes) {
    continentes.add(Continente(nombre: conti["nombre"], imagen: conti["imagen"]));
  }

  return continentes;
}

// Implementar el método getPaises a partir del continente
List<Pais> getPaisesNuevo(String nombreContinente){
  List<Pais> paises = [];

  // Recorremos la lista de provincies en RepositoryData para encontrar la que se busca
  for (var conti in apiNueva.continentes) {
    if (conti["nombre"] == nombreContinente) {
      // Cuando encontramos el continente, reocrremos los paises
      // y añadimos a la lista paises con los datos de cada pais

      for (var paisJSON in conti["paises"]) {
        paises.add(Pais.fromJSON(paisJSON));
      }
    }
  }

  return paises;
}

Pais infoPaisNuevo(String nombrePais) {
for (var conti in apiNueva.continentes) {
    for (var paisJSON in conti['paises']) {
      if (paisJSON['nombre'] == nombrePais) {
        return Pais.fromJSON(paisJSON);
      }
  }
}
return Pais(nombre: "País no encontrado");
}
```

Añadimos las funciones correspondientes.

Fromulario

```
class _FormScreenState extends State<FormScreen> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final String correctEmail = "pepe@yahoo.es";
  final String correctPass = "pepeyahoo";
```

Creamos una pantalla Formulario, añadimos una clave global para el formulario.

_emailController y _passwordController seran los controladores de dichos campos del formulario.

Debajo están declaradas las constantes con el email y contraseña correctos.

```
Expanded creaCampoEmail() {
  return Expanded(
    child: TextFormField(
      validator: (value) {
        if (value?.isEmpty ?? true) {
          return 'No puede estar vacío';
        }
        final regexCorreo = RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');

        // Validación del correo con la expresión regular
        if (!regexCorreo.hasMatch(value ?? "")) {
          return 'La dirección de correo no es válida';
        }
        return null;
      },
      controller: _emailController,
      keyboardType: TextInputType.text,
      decoration: InputDecoration(
        border: OutlineInputBorder(borderRadius: BorderRadius.circular(20)),
        icon: const Icon(Icons.email),
        labelText: 'Correo Electrónico',
      ), // InputDecoration
    ), // TextFormField
  ); // Expanded
}
```

La creación del campo incluye las validaciones, como no poder estar vacío, y que cumpla la expresión regular.

En el campo contraseña el unico cambio es que no hay expresión regular y se añade obscureText: true, obscureText: true para ocultar el texto de la contraseña

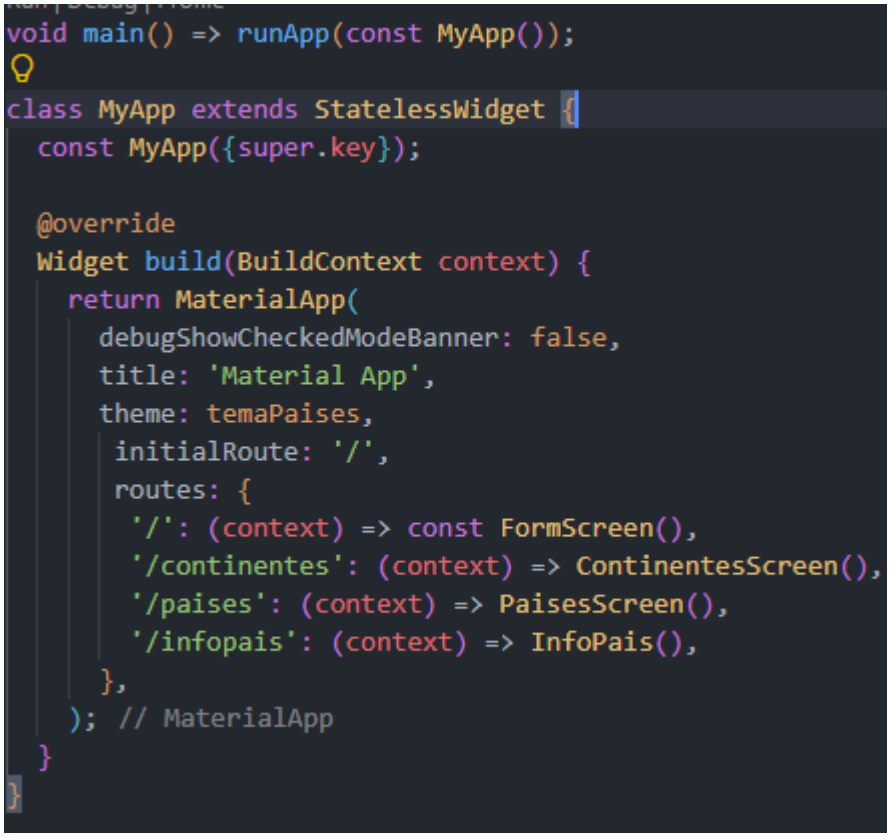
```
ElevatedButton creaBotonSubmit() {
  return ElevatedButton(
    onPressed: () {
      if (_formKey.currentState?.validate() ?? false) {
        if (_emailController.text == correctEmail &&
            _passwordController.text == correctPass) {
          Navigator.pushNamedAndRemoveUntil(
            context,
            '/continentes',
            (Route<dynamic> route) => false,
          );
        } else {
          showDialog(
```

En el botón haremos que se validen los formularios, si la validación es correcta, comprobamos que el texto en los campos sea igual que el que tenemos definido como correcto; si es así, iremos a continentes; si no, mostramos el dialog

```
showDialog(
  context: context,
  builder: (context) {
    return AlertDialog(
      title: const Text("Error"),
      content: const Text("El usuario o la contraseña son incorrectas."),
      actions: [
        TextButton(
          onPressed: () {
            Navigator.pop(context, "Volver");
          },
          child: const Text("Volver"),
        ), // TextButton
        TextButton(
          onPressed: () {
            _emailController.text = correctEmail;
            _passwordController.text = correctPass;
            Navigator.pop(context, "Volver");
          },
          child: const Text("Rellenar usuario"),
        ), // TextButton
      ],
    ); // AlertDialog
```

En el dialog tendremos 2 botones, volver, que devuelve al formulario con los campos como están. Y el botón rellenar que completa los campos con el usuario y contraseña correctos.

Enrutado



```dart
void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Material App',
      theme: temaPaises,
       initialRoute: '/',
       routes: {
        '/': (context) => const FormScreen(),
        '/continentes': (context) => ContinentesScreen(),
        '/paises': (context) => PaisesScreen(),
        '/infopais': (context) => InfoPais(),
      },
    ); // MaterialApp
  }
}
```

En el main declaramos la navegación con rutas, en lugar de home, pondremos initialRoute: '/', en routes declaramos todas las rutas con sus pantallas.

Navegación entre pantallas

Para poder regresar a la página anterior tendremos que declarar un appbar

```
return Scaffold(
  appBar: AppBar(
    title: Text("Paises de $nombreContinente"),
  ), // AppBar
```

Los elementos como continentes y países deben ser clickables y navegables, para ello usaremos inkwell

```
return Material(
 child: InkWell(
   onTap: () {
     // Navegamos a la ruta '/paises'
     // Enviamos a través de 'arguments' el nombre de este pais específico
     Navigator.pushNamed(
       context,
       '/infopais',
       arguments: nombre,
     );
   },
```

En onTap haremos la navegación en la que le pasaremos un argumento, como el nombre del continente o país.

Dicho argumento lo recuperaremos con la siguiente línea.

```
final String nombreContinente =
    ModalRoute.of(context)?.settings.arguments as String? ?? 'Europa';
```

NavigationBar

Para usar el navigationBar en infopais empezaremos declarando las pantallas

```
final List<Widget> pantallas = [
  InfoPaisGeneral(nombrePais),
  InfoPaisDetalle(nombrePais),
];
```

Y deberemos usar un scaffold

```
return Scaffold(
  body: pantallas[_currentIndex],
  bottomNavigationBar: NavigationBar(
    selectedIndex: _currentIndex,
    onDestinationSelected: (int index) {
      setState(() {
        _currentIndex = index;
      });
    },
    destinations: const [
      NavigationDestination(
        icon: Icon(Icons.info_outline),
        selectedIcon: Icon(Icons.info),
        label: 'General',
      ), // NavigationDestination
      NavigationDestination(
        icon: Icon(Icons.wb_sunny_outlined),
        selectedIcon: Icon(Icons.sunny),
        label: 'Detalle',
      ), // NavigationDestination
    ],
  ), // NavigationBar
); // Scaffold
```

El body tendrá que ser la lista de pantallas con el indice actual.

Con el body así podemos añadir bottomNavigationBar y añadirle el widget,

En el apartado destinations elegiremos los iconos y etiqueta de la navegación, que al pulsarla actualizara el indice para cambiar de pantalla.

Ampliación de la práctica

Navigation bar en paises

Añadiremos navigation bar para cambiar entre lista y grid en paises

```
return Scaffold(
  body: pantallas[_currentIndex],
  bottomNavigationBar: NavigationBar(
    selectedIndex: _currentIndex,
    onDestinationSelected: (int index) {
      setState(() {
        _currentIndex = index;
      });
    },
    destinations: const [
      NavigationDestination(
        icon: Icon(Icons.view_list_outlined),
        selectedIcon: Icon(Icons.view_list),
        label: 'Lista',
      ), // NavigationDestination
      NavigationDestination(
        icon: Icon(Icons.grid_view_outlined),
        selectedIcon: Icon(Icons.grid_view),
        label: 'Grid',
      ), // NavigationDestination
    ],
  ), // NavigationBar
); // Scaffold
```

Para ello creamos una pantalla países en la que pondremos un navigationbar con las pantallas siguientes

```
final List<Widget> pantallas = [
  PaisesListScreen(nombreContinente),
  PaisesGridScreen(nombreContinente),
];
```

Habrá que modificar las pantallas para que reciba por parámetro el continente que la pantalla países recibirá como argumento de navegación

Orientación de pantalla

Para comprobar la orientación de la pantalla añadiremos la siguiente línea

```
final bool isLandscape =
    MediaQuery.of(context).orientation == Orientation.landscape;
```

Una vez sabemos si es vertical u horizontal, podemos manejar que hacer en cada caso.

```
if (!isLandscape) {
  return Scaffold(
    body: pantallas[_currentIndex],
    bottomNavigationBar: NavigationBar(
      selectedIndex: _currentIndex,
      onDestinationSelected: (int index) {
        setState(() {
          _currentIndex = index;
        });
      },
      destinations: const [
        NavigationDestination(
          icon: Icon(Icons.view_list_outlined),
          selectedIcon: Icon(Icons.view_list),
          label: 'Lista',
        ), // NavigationDestination
        NavigationDestination(
          icon: Icon(Icons.grid_view_outlined),
          selectedIcon: Icon(Icons.grid_view),
          label: 'Grid',
        ), // NavigationDestination
      ],
    ), // NavigationBar
  ); // Scaffold
```

Si no está apaisado, mantenemos el navigationBar

```
} else {
  return Scaffold(
    body: Row(
      children: [
        NavigationRail(
          selectedIndex: _currentIndex,
          onDestinationSelected: (int index) {
            setState(() {
              _currentIndex = index;
            });
          },
          labelType: NavigationRailLabelType.all, // Muestra los textos debajo de los iconos
          destinations: const [
            NavigationRailDestination(
              icon: Icon(Icons.view_list_outlined),
              selectedIcon: Icon(Icons.view_list),
              label: Text('Lista'),
            ), // NavigationRailDestination
            NavigationRailDestination(
              icon: Icon(Icons.grid_view_outlined),
              selectedIcon: Icon(Icons.grid_view),
              label: Text('Grid'),
            ), // NavigationRailDestination
          ],
        ), // NavigationRail
        const VerticalDivider(
          thickness: 1,
          width: 1,
        ), // Línea divisoria vertical opcional // VerticalDivider
        // El Expanded es obligatorio para que la pantalla ocupe todo el espacio restante
        Expanded(child: pantallas[_currentIndex]),
      ],
    ), // Row
  ); // Scaffold
```

Si está apaisado usaremos un navigationRail, el funcionamiento es prácticamente el mismo que el de navigationBar pero añadiremos el tipo de etiqueta y una barra vertical para separar.